

CITS3011 Intelligent Agent Project Report

Lewei Xu (23709058)

Basic Techniques

1. Internal State Representation

This was the first technique/idea that I implemented that is the foundation of all the other techniques described below. Inspired by the internal state graph implemented in **GreedyAgent**, I adapted this graph that uses the **networkx** library to include various other information in its nodes and edges. The implementation of this internal state representation is contained within the lines 36-85 in **agent_23709058.py**, and contains 2 functions: **build_state_graph()** and **update_state_graph()**.

build_state_graph()

- This function is used to construct the internal state representation of the diplomacy game, and is called once within **new_game()**.
- We first construct the nodes of the graph using **game.map.loc_type** which is a dictionary of locations and the locations type. With this information, we can initialize each node as a location, with 2 attributes **type** which corresponds to the unit type (WATER, LAND or COAST) and **units** which is a 2-tuple storing the type of unit (A or F) and the power that unit belongs to.
- Next we use **game.map.abuts()** method to construct the edges between the nodes. For each pair of nodes in the state graph, we test if an army and a fleet can move between the nodes, and if it can, we add the edge and set the corresponding attribute **army** and/or **fleet** to True. For example, to test if an army can move between 2 nodes:

```
if self.game.map.abuts('A', i, '-', j):
    state_graph.add_edge(i, j, army=True, fleet=False)
```

update_state_graph()

- This function is used to update the internal state representation every phase, and is called within **update_game()** after the game engine processes the new moves.
- First, we initialize (and reset) a couple more attributes for each node, these include **is_supply** which is a boolean indicating whether a node is a supply center, and **owner** which is a string containing which power current owns the supply center (if it is one).
- Finally, we use the diplomacy library's game object to find which locations are supply centers, where each unit is located, and who owns each supply center, then update the internal state representation with this information.

This internal state representation of the game is crucial for the other techniques implemented, as it allows for conventional BFS, DFS and other graph traversal algorithms to be directly used for choosing the orders that are to be given to each unit. Although all this information is contained within the diplomacy game object, it is not easily usable, and we would need to write custom algorithms to find information such as shortest paths between 2 locations.

At this stage, there is no real experimental data regarding agent performance to be provided, however, previously when directly using the game object to get information regarding the current phase, it was found that there were no real demerits in terms of time complexity when constructing and constantly updating an internal state representation. This is due to the fact that all operations runs in $O(n)$ time anyway, so constructing an internal state representation will only serve to make other techniques more easy to implement.

2. Case-Based Reasoning

After playing webdiplomacy against bots for some time, I designed a heuristic that describes a number of cases that a good player would generally make. These cases utilize the internal state representation described above to work cleanly. The implementation is contained within the `get_actions()` method. Initially there were only 4 cases:

- Case 1: for every unit that is currently in a supply center that is not yet owned by us, hold. This generally applies to when a unit has moved into an unoccupied supply center in the spring phase, and we do not yet own the supply center, so we would hold and wait for the fall phase so that we can build more units during build phase.
- Case 2: for remaining units, check neighboring nodes for unoccupied supply centers and move there. Building more units should always be a priority, and to build more units, we need to capture more supply centers.
- Case 3: for remaining units, if >2 units are adjacent to a location occupied by an enemy unit, move one unit to that location and have all other units support the move/attack. This case is mainly to simulate an aggressive agent that will always try to take space if it can.
- Case 4: Move all remaining units that haven't yet been given an order closer to their closest supply center not owned by us via the shortest path. This is so that there is a higher chance for Case 2 and Case 3 triggering in the next phase.

The above cases outline how the agent works at a high level. Later on, more cases were added to fine tune the behavior of the agent:

- Case 0: if in a build phase, build a random unit in a home supply center
- Case 0.5: if in a retreat phase, retreat to a random adjacent location if possible, else disband
- Case 2.5: an extension of Case 2 that also checks for convoy options
- Case 3.5: move any fleets into a WATER location if possible to vacate space for army units and also allow for convoys

This case-based reasoning is meant to simulate a relatively beginner human playing diplomacy. Due to the nature of Scenario 1, I went with this approach to achieve a 100% win rate in Scenario 1 where all other agents are static. At the same time, as this agent is versus other agents, the relatively aggressive heuristic should also work quite well against other simple agents.

With just cases 1-4 + case 0 that allows for building more units, the agent was able to achieve a relatively high win rate in scenario 1 of $>70\%$. However, when implementing case 0.5, 2.5 and 3.5 which was added after visualizing a couple games where the agent failed to win, the agent managed to achieve a 100% win rate in scenario 1. Next, I tested this agent in scenario 2 with 20 iterations per power, and found that it actually achieved an approximately 25% win rate with around 7 supply centers captured on average. This shows that the aggressive agent worked well against other agents.

New Techniques

1. Dynamic Supply Center Targeting

This technique was mainly focused on improving Case 4 which controls how all remaining units move after the priority moves have been placed (aggressively moving to supply centers and attacking enemy units). In Case 4, we now define a function `get_target()` that selects a random unit, then finds the closest location to that unit that contains an enemy supply center. We then try to move all units towards that supply center so that as many units are adjacent to that target supply center location as possible. We also defined a class-level variable `RETRY_COUNT` that specifies how many phases we will try to capture that target. If after `RETRY_COUNT` is reached and that supply center still has not been captured, we add that supply center location to the `failed_targets` variable and call `get_target()` again to select a new target. Note that Case 3 runs before Case 4, so all attacks are handled before Case 4 is reached.

Although the above sounds fairly simple in theory, there were various problems that needed to be ironed out. One of the biggest problems with this technique is that the logic to move units closer to the target

involves using BFS to find the shortest path towards the target. This means if there were originally no units bordering the target, then only one unit would end up bordering that target as all units try to move along a predefined path, resulting in no way for that adjacent unit to receive support and attack. To work around this problem, we adapted the BFS into the function `get_closest_unoccupied` that returns a tuple (location, fleets) where location is the closest location to the target that is unoccupied. If location is only reachable via convoy, fleets is a list of fleets involved in the convoy. If none are found, it returns the closest adjacent location on the shortest path to target, even if occupied, in the off chance that the unit originally in that location moves out of for whatever reason. In the main logic of Case 4, we try to move all units that aren't yet adjacent to the target closer towards the target, then finally we try to move any units already adjacent to the target to another location still adjacent to the target. This massively increases the chance for an attack with multiple other units for support when Case 3 is run in the next phase.

After this was implemented, we immediately saw a large increase in overall win rate for Scenario 2 to approximately 40-45%. Results were fairly consistent, and upon viewing how the agent performed visually, we saw that the dynamic targeting of enemy supply centers would often overwhelm other agents. Despite the agent often leaving its back exposed and leaving supply centers undefended, this blitzkrieg tactic that captured enemy supply centers aggressively makes up for it. By moving all units away from the home supply centers and capturing more supply centers, it allowed the agent to build more units in its home supply centers anyway and defend that way.

2. Power-Specific Strategic Variation

After implementing the basic techniques, the following 2 cases were improved by implementing power-specific strategic variation.

- Case 0: in the logic that builds new units, the variable `fleet_ratio_threshold` was added that calculates the proportion of fleet units compared to the total number of units, and set a different threshold for each power. E.g. for England, `fleet_ratio_threshold` is set to 0.4 meaning that England should maintain 40% of its units as fleets, whilst for powers such as Germany, `fleet_ratio_threshold` is set very low to 0.1.
- Case 2: this case was changed purely to increase the win rate of the agent when it is playing as England. Case 2 makes it so that the agent will try to move units to unoccupied supply centers as a first priority, but in the case of England, we want to prioritise convoying armies to the mainland as early as possible. To achieve this, we set the variable `self.english_wait_fleets` that is a 2-tuple containing the start and end phases where we do not want England's fleets to move to coasts that contain supply centers. When this action is blocked, Case 2.5 and 4 gets triggered as the England's fleets have not yet been given orders.

Although the above 2 basic techniques achieved fairly good overall performance, we notice that when the agent had to play as England or Italy, it consistently underperformed compared to when playing other powers. This was due to their high reliance on fleets and control of WATER locations. To improve the win rates of each power (and in turn the overall win rate), we experimented with different fleet ratio thresholds for each power and settled on the values that can be seen in Case 0 in `get_actions()`.

After implementing this technique, including the dynamic supply center targeting, the win rates of England and Italy increase by about 10% on average. Interesting, when `test.py` is run multiple times on Scenario 2 with 25 iterations per power multiple times, the results would vary. Sometimes, the win rate for England is very low at around 18%, while other times it can be as high as 50%. On average however, we did see an increase in overall win rate in Scenario 2 to be consistently above 45%. Running multiple tests with different iterations per power, the overall win rate did vary per run, ranging from 45%-60%, however the overall win rate virtually never dropped below 45%.